Prof. Dr. Ghassan Karame
M. Sc. Simon Lachnit
M. Sc. Annika Wilde
M. Sc. Pascal Zimmer

Chair for Information Security
System Security

Summer semester 2026

Task 5

Software Security - More Advanced Overflows

Reward:	15 Points
Mode:	Project / Group work (max. 3 students)
Due:	13.07.2025 -- 14:15

Information

- **Remote Exercise Framework (REF)**

  Before you start the task, you should read, understand, and follow the instructions for the *Remote Exercise Framework* provided in Moodle.

- **Group Work**

  This task can be completed in groups. You may work in a group of up to 3 students (i.e., 1, 2, or 3 people), and your group selection is binding for the remainder of the semester. It is sufficient if one person from the group submits the solution (both the code in REF and a submission PDF in Moodle).

- **Deliverables**

  All code submissions are made directly in REF; you can find more details in the REF guide on Moodle. All code must be sufficiently and meaningfully commented. Solutions to pen & paper tasks are uploaded as a single PDF file to Moodle.

  Important: We reserve the right to check all submissions for plagiarism and take appropriate action.

Task 6: Stack Canaries

After the systems security lecture, one of your classmates realized that their program is vulnerable to a buffer overflow. Fortunately, they just learned about stack canaries as a countermeasure! Distrustful of existing implementations, they decided to create their own version to ensure the canaries actually work. You managed to steal parts of the source code — but unfortunately the valuable canary values are missing. Can you still exploit the program?

Write a (commented) script (`solution.sh`) that prints the exploit to `stdout` so that the output can be used as an argument for the target program. Once a `/bin/sh` shell is started, you should be able to retrieve the flag in the same directory. Since the countermeasure is non-deterministic, you may need to run `task submit` multiple times.

```
#include <stdlib.h>
#include <stdio.h>
#include <string.h>
#include <time.h>

int stack_canaries[10] = ...
int r;

int overflow(char *password) {
    int canary = stack_canaries[r];
    char password_buffer[16];
    strcpy(password_buffer, password);
    if (canary != stack_canaries[r]) {
        printf("Someone␣tampered␣with␣my␣stack␣:(\n");
        printf("I'm␣out!\n");
        exit(1);
    }
    return 0;
}
int main(int argc, char *argv[]) {
    if(argc < 2) {
        printf("Usage:␣%s␣<password>\n", argv[0]);
        exit(0);
    }
    srand(time(NULL));
    r=rand() % 10;
    overflow(argv[1]);
}
```

Notes:

- *Don't overthink — this task uses only a simple, conceptual simulation of stack canaries.*
- *Keep in mind that there are various "bad characters" (aside from null bytes).* *http://blog.disects.com/2014/04/exploitation-identifying-bad-characters.html*

Task name for remote login: `fake_canary`

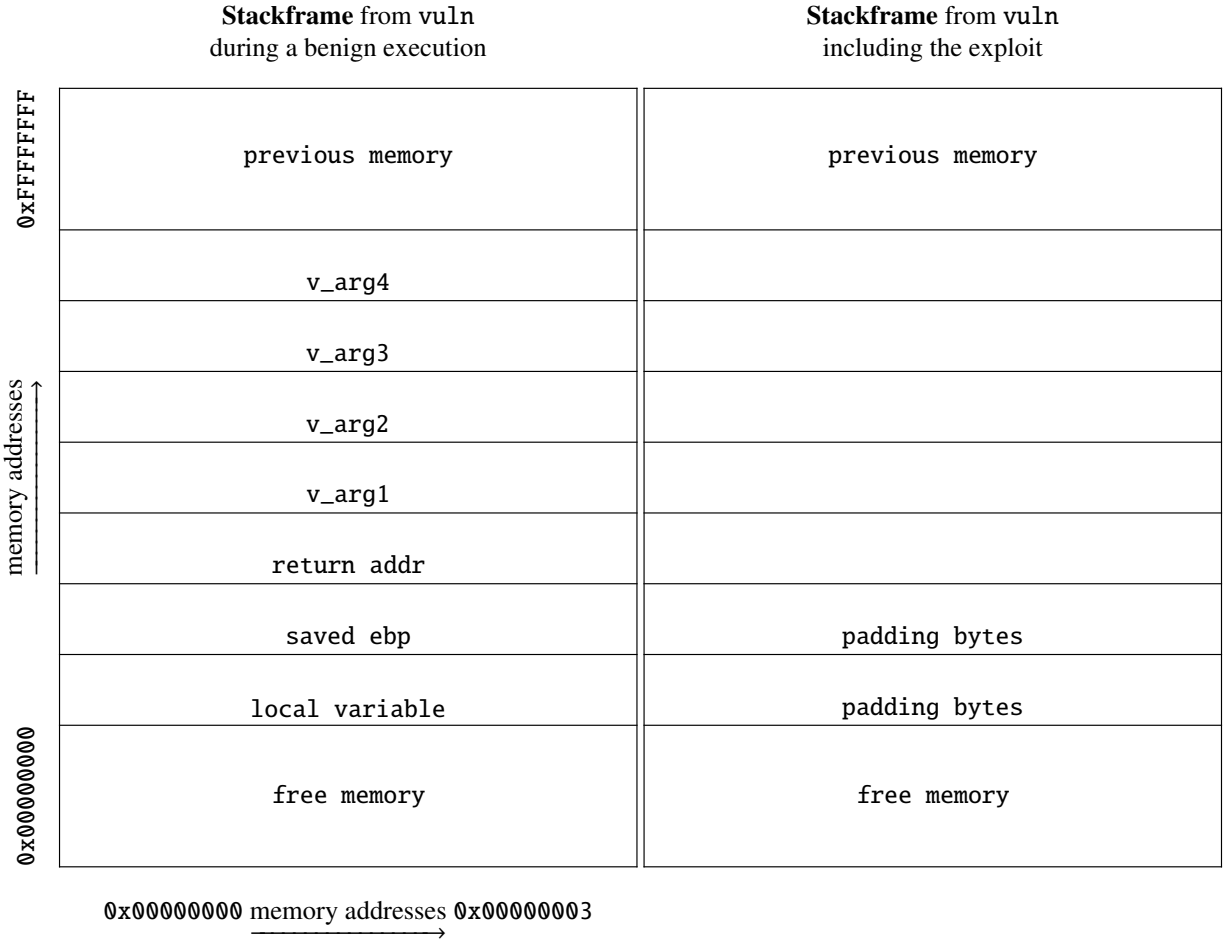
Task 7: Ret2Libc

The idea behind *Data Execution Prevention (DEP)* is to mark memory pages as either executable (for code) or writable (but not executable). Injected shellcode on the stack cannot be executed, as those pages are marked writable. An attacker can bypass this protection by *reusing* existing program code instead of injecting new code. One well-known technique is *return-to-libc*: we fake a function call to the standard library `libc` (linked into all C/C++ programs) by spoofing the call and redirecting control flow to the library.

a) **Return-To-Libc — Theory**

In a return-to-libc attack, multiple functions can be executed sequentially. Assume we control a stack-based buffer overflow in the function `vuln`, allowing us to overwrite the stack frame.

i) Prepare the stack frame for function `vuln` so that the library function `f` executes first, followed by function `g`.



You know the following about `f` and `g`:

- `f` is located at address `f_addr` and expects one parameter (label: `f_arg1`)
- `g` is at address `g_addr` and expects two parameters (labels: `g_arg1, g_arg2`)

ii) Would it be problematic if `f` expected more than one parameter? Does this limitation also apply to `g`? Justify your answer.

iii) What problem arises when trying to call a third function after `f` and `g`? Justify your answer.

b) **Return-To-Libc — Practice**

Now we want to exploit the `ret2libc` program, which is vulnerable to a stack-based buffer overflow. Write a commented script for each of the two tasks that prints the exploit to stdout, so the output can be used as an argument for the target program.

i) Launch a shell via a return-to-libc attack: exploit the vulnerability in the binary to call the `system(·)` function in `libc` with the argument `/bin/sh`. Write your solution in the file `solution.sh`.

ii) `system(·)` can be used to run any command, not just `/bin/sh`. To demonstrate this, write an exploit that creates a file named `owned` in the directory `t0p_s3cr3t`. Note that strings can be injected into the process by setting environment variables before execution. Ensure your exploit terminates cleanly by calling `exit()` after `system(·)`. You can specify the score you want to receive for Task 8 by writing a number from 0–100 into the file using your exploit ;) Write your solution in the file `solution_grade.sh`.

Task name for remote login: `ret2libc`

Hints:

- *In GDB, use the command* `grep <str>` *to search for strings and find their addresses.*
- *GDB also allows retrieving function addresses using* `print <func_name>`*.*
- *Is there a difference between* `"command"` *and* `" command"`*?*

Plagiarism & Use of Generative AI

In this course, we respect other people's work and do not tolerate plagiarism. That means you should always give credit to the original authors if you:

- use someone else's work (whether you copy it directly, almost directly, or rewrite it in your own words),
- reuse parts of someone else's work, such as equations, figures, or tables.

We strongly encourage you to work on this assignment without using generative AI tools (like large language models), so you can build your own understanding of the material. Please, **in all cases**, state clearly how you used such tools:

- If you only used them for small edits (e.g., grammar or wording), add a note like: ''This assignment was edited for grammar using [Tool Name].''
- If you did not use any AI tools, add: ''This assignment was completed without the use of AI tools.''
- If you use AI for more than small edits, please specify in detail which parts of the solutions are generated by the AI tool and which steps you took to ensure the correctness of the output.
- Work that relies on AI to generate the major part of the submission, e.g., full scripts or key parts of the solution, will not be accepted.

If you use Stack Overflow or similar resources, make sure to clearly cite any code you did not write yourself.